

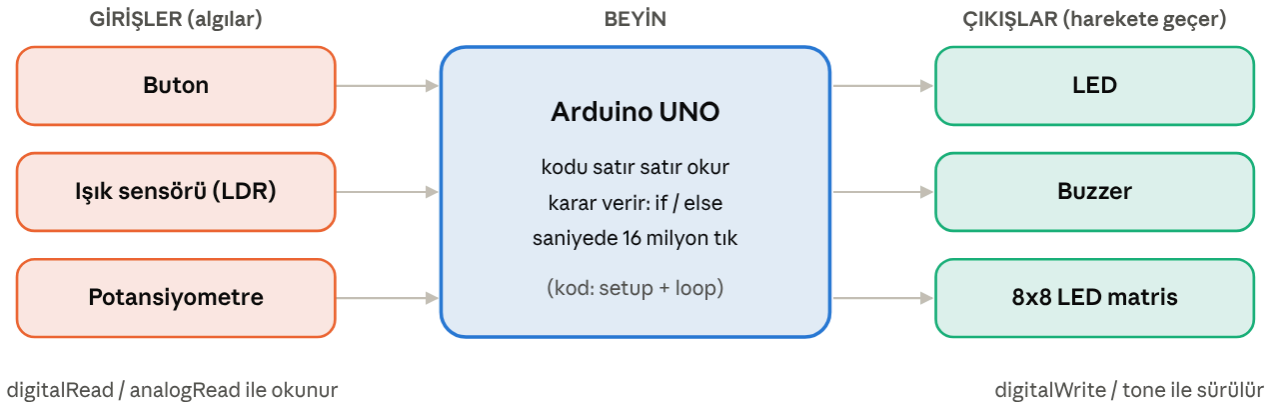
Arduino Temelleri

Oct 8, 2026 · @Arif

1. Arduino nedir?

Arduino, yazdığımız kodu çalıştıran küçük bir bilgisayardır. Dışarıdan bilgi alır (buton basıldı mı?), koda göre karar verir ve bir şeyi harekete geçirir (LED'i yak).

Arduino bir beyin gibi çalışır: algılar, karar verir, harekete geçer



giriş → beyin → çıkış

Kartın ortasındaki siyah çip **ATmega328P** mikrodenetleyicisidir; asıl beyin odur. Kartın geri kalanı bu çipe elektrik verir, USB'den kod yüklemeyi sağlar ve bacaklarını kablo takılabilecek deliklere taşır.

Özellik	Arduino UNO	Ne demek?
Hız	16 MHz	Saniyede 16 milyon kez "tık"lar; her tıkta çok basit bir iş yapar
Program hafızası (Flash)	32 KB	Yüklediğin kod burada durur, elektrik kesilse de silinmez
Çalışma hafızası (RAM)	2 KB	Değişkenler burada durur, elektrik kesilince silinir
Kalıcı not defteri (EEPROM)	1 KB	Rekor gibi saklanması gereken küçük veriler için

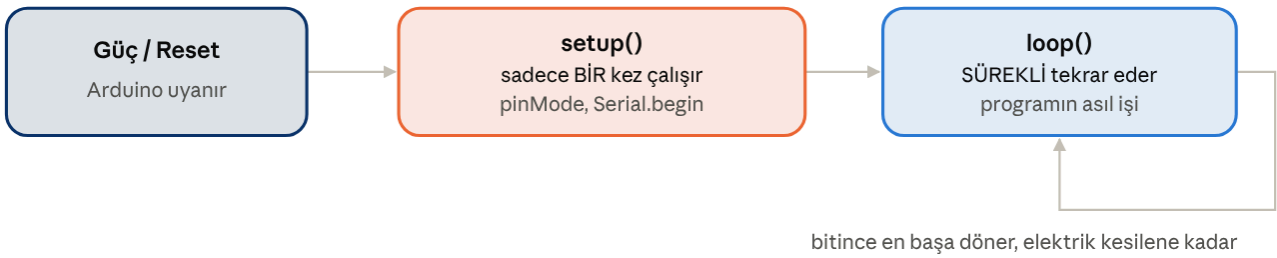
2 KB RAM çok azdır: bir telefonun RAM'i bunun yaklaşık 4 milyon katı. Bu yüzden Arduino'da değişken tipini doğru seçmek önemlidir (7. bölüm).

Kod nasıl Arduino'ya gidiyor? Arduino IDE'de yazdığımız C++ kodu önce **derlenir**: bilgisayar onu çipin anladığı 0 ve 1'lere çevirir. Sonra USB kablosuyla çipin Flash hafızasına yazılır. "Doğrula" butonu sadece derler (hata var mı diye bakar), "Yükle" butonu derler ve gönderir.

2. Programın iskeleti: setup() ve loop()

Her Arduino programında bu iki fonksiyon olmak zorunda. `setup()` hazırlık yapar, `loop()` asıl işi durmadan tekrarlar.

setup() bir kez, loop() sonsuza kadar çalışır



bir Arduino programının hayatı

```

void setup() {
  pinMode(13, OUTPUT);    // hazırlık: 13. pin çıkış olsun
}

void loop() {
  digitalWrite(13, HIGH);  // LED'i yak
  delay(1000);             // 1 saniye bekle
  digitalWrite(13, LOW);   // LED'i söndür
  delay(1000);             // 1 saniye bekle
}                          // ...ve en başa dön
  
```

Benzetme: Sabah kalkınca bir kez dişlerini fırçalarsın, çantanı hazırlarsın (setup). Sonra kalbin gün boyunca durmadan atar (loop).

Kod yazmanın 5 kuralı

1. **Her komut ; ile biter.** Noktalı virgül, cümle sonundaki nokta gibidir.
2. **Süslü parantez { }** bir **bloğu kapsar**. Açılan her { kapanmalı. IDE'de bir parantezin yanına tıklayınca eş parlar.
3. **Büyük/küçük harf önemlidir.** `digitalWrite` çalışır, `digitalwrite` çalışmaz.
4. **// sonrası yorumdur.** Arduino okumaz, insanlar için yazılır. Birden çok satır için `/* ... */` kullanılır.

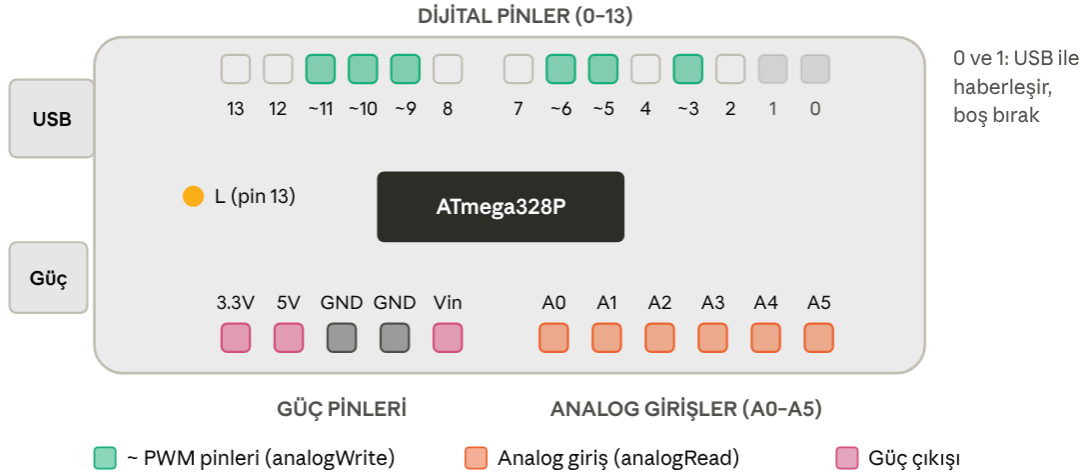
5. Kod yukarıdan aşağı sırayla çalışır. Bir satır bitmeden bir sonrakine geçilmez.

Perde arkası: Arduino aslında gizli bir `main()` fonksiyonu çalıştırır. Bu fonksiyon önce `setup()` 'ı bir kez çağırır, sonra `while (true) { loop(); }` ile `loop()` 'u sonsuza kadar çağırır. Biz bu kısmı görmeyiz, IDE bizim için ekler.

3. Pinler: Arduino'nun kolları ve kulakları

Pin, Arduino'nun dış dünyaya kablo bağladığımız deliğidir. Her pinin bir numarası vardır; kodda pinlere bu numarayla sesleniriz.

Arduino UNO'nun pinleri üç gruptur



Arduino UNO · basitleştirilmiş üstten görünüş

Grup	Pinler	Ne yapar?
Dijital	0-13	Sadece iki durum bilir: açık (HIGH, 5 V) ya da kapalı (LOW, 0 V). LED yakmak, buton okumak için
PWM (~ işaretli)	3, 5, 6, 9, 10, 11	Dijital pinlerin özel olanları. Çok hızlı açıp kapanarak "yarım" parlaklık taklidi yapar (5. bölüm)
Analog giriş	A0-A5	0 ile 5 V arasındaki her gerilimi ölçer, 0-1023 arası bir sayıya çevirir. Gerekirse dijital pin gibi de kullanılır
Güç	5V, 3.3V, GND, Vin	Devreye elektrik verir. GND, devrenin "eksi"si ve ortak dönüş yoludur

Önemli kurallar:

- 0 ve 1 numaralı pinleri boş bırak.** Arduino bilgisayarla bu pinler üzerinden konuşur. Bunlara bir şey bağlarsan kod yüklenmeyebilir ve Seri Monitör bozulur.

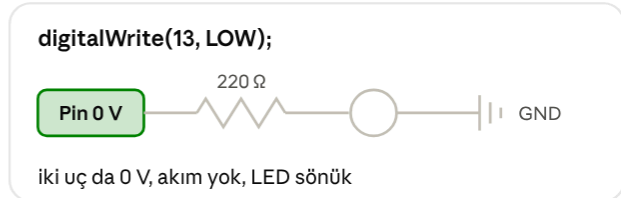
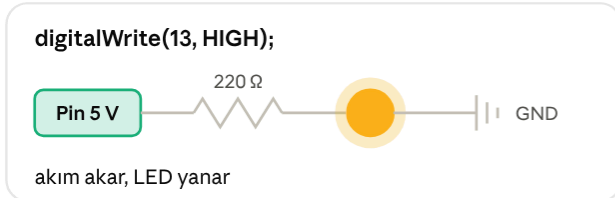
- **Bir pin en fazla 20 mA rahatça verebilir** (mutlak sınır 40 mA). Bir LED için yeterli. Ama motor gibi güçlü bir şeyi doğrudan pine bağlama; pin yanar.
- **LED'ler hep dirençle bağlanır** (genelde 220 Ω). Direnç, akımı sınırlayan bir "musluk"tur.
- **Her devrede GND ortaktır.** Akımın geri döneceği bir yol olmalı; GND bağlanmazsa hiçbir şey çalışmaz.
- Kartta 13. pine bağlı küçük bir L LED'i vardır. Hiç kablo takmadan ilk denemeyi bununla yapabilirsin.

4. pinMode, digitalWrite, digitalRead

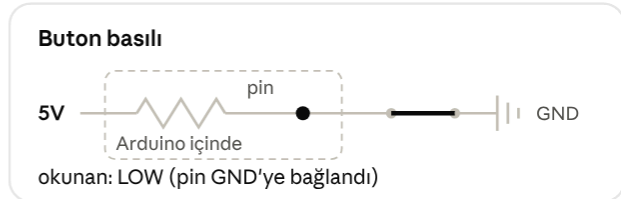
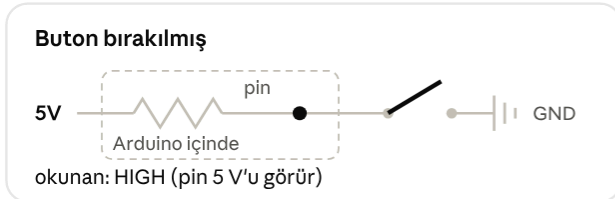
Bir pin ya **konusur** (çıkış: LED'e elektrik verir) ya da **dinler** (giriş: butonu okur). Hangisini yapacağını `pinMode` ile `setup()` içinde söyleriz.

HIGH = 5 V, LOW = 0 V: LED yakarken de buton okurken de

ÇIKIŞ (OUTPUT): Arduino gerilim VERİR



GİRİŞ (INPUT_PULLUP): Arduino gerilimi OKUR



çıkış ve giriş · dört durum

pinMode(pin, mod)

Mod	Pin ne yapar?	Örnek
OUTPUT	Konusur: 5 V ya da 0 V verir	LED, buzzer, matris
INPUT	Dinler. Ama pin boşta kalırsa "havada" kalır ve rastgele okur	Dış dirençli buton, sensör
INPUT_PULLUP	Dinler. İçerideki direnç pini 5 V'a çeker; boştayken kesin HIGH okur	Buton (tek bacak pine, diğeri GND'ye)

`pinMode` yazılmazsa pin varsayılan olarak `INPUT` olur. LED'i `pinMode` olmadan yakmaya çalışırsan çok sönük yanar ya da hiç yanmaz. Bu, derste en sık görülen hatalardan biridir.

digitalWrite(pin, HIGH / LOW)

Pini açar (5 V) ya da kapatır (0 V). Bir ışık anahtarı gibi düşünebilirsin. Akım her zaman yüksek gerilimden alçağa akar: pin 5 V, GND 0 V ise LED'in üzerinden akım geçer.

digitalRead(pin)

Pindeki gerilime bakar ve **HIGH** (1) ya da **LOW** (0) döndürür. Kabaca 3 V'un üstü HIGH, 1,5 V'un altı LOW sayılır.

```
void setup() {
  pinMode(13, OUTPUT);
  pinMode(A4, INPUT_PULLUP);    // buton: A4 ve GND arasında
}

void loop() {
  if (digitalRead(A4) == LOW) { // basılıysa (ters mantık!)
    digitalWrite(13, HIGH);
  } else {
    digitalWrite(13, LOW);
  }
}
```

Neden INPUT_PULLUP'ta mantık ters?

Resmin alt sırasına bak. Buton bırakılmışken pin, içerideki direnç sayesinde 5 V'u görür ve **HIGH** okur. Butona basınca pin doğrudan GND'ye bağlanır ve **LOW** okur. Yani "basılı = LOW".

Peki direnç olmasa ne olurdu? Buton bırakırken pin hiçbir yere bağlı olmazdı. Havada kalan bir pin, elinin yakınlığından bile etkilenip rastgele HIGH ve LOW okur. Direnç pini "varsayılan olarak HIGH" tutar; buna **pull-up** (yukarı çekme) denir.

5. analogRead ve analogWrite

Dijital dünyada sadece iki seçenek var: açık ya da kapalı. Gerçek dünyada ise ışık, ses ve sıcaklık kademe kademe değişir. Bu iki komut aradaki değerlerle çalışmayı sağlar.

analogRead(pin): gerilimi sayıya çevirir

A0–A5 pinlerindeki 0–5 V arası gerilimi ölçer ve 0–1023 arası bir sayı verir. Neden 1023? Arduino'nun ölçü aleti 10 bitliktir: $2^{10} = 1024$ farklı değer (0'dan 1023'e).

Pindeki gerilim	analogRead sonucu
0 V	0
1,25 V	yaklaşık 256
2,5 V	yaklaşık 512
5 V	1023

Her adım yaklaşık $5 \text{ V} / 1024 \approx 4,9 \text{ mV}$ 'tur. Potansiyometre (ayarlı direnç) çevrildikçe ya da LDR'ye düşen ışık değiştikçe bu sayı değişir.

analogWrite(pin, 0-255): "yarım" güç taklidi

Arduino gerçekte 2,5 V üretemez; sadece 0 ya da 5 V verebilir. Bunun yerine pini çok hızlı açıp kapatır. Pin zamanın yarısında açıksa LED, gözümüze yarı parlaklıkta görünür. Bu yöntem **PWM** (Pulse Width Modulation, darbe genişliği ayarı) denir. Sadece ~ işaretli pinlerde çalışır: 3, 5, 6, 9, 10, 11.

analogWrite, LED'i çok hızlı açıp kapatarak kısar

KOD

PİNDEKİ GERİLİM: üst = 5 V, alt = 0 V

GÖZÜN GÖRDÜĞÜ

analogWrite(9, 0);
açık kalma süresi: %0



analogWrite(9, 64);
açık kalma süresi: %25



analogWrite(9, 191);
açık kalma süresi: %75



analogWrite(9, 255);
açık kalma süresi: %100



her dalga yaklaşık 2 ms: saniyede ~490 kez açılıp kapanır, göz bunu fark etmez

PWM · dört farklı analogWrite değeri

Dikkat: iki komutun aralığı farklı

`analogRead` 0-1023 verir ama `analogWrite` 0-255 ister. Potansiyometreyle LED parlaklığı ayarlamak için sayıyı çevirmek gerekir. Bunun için `map` fonksiyonu kullanılır:

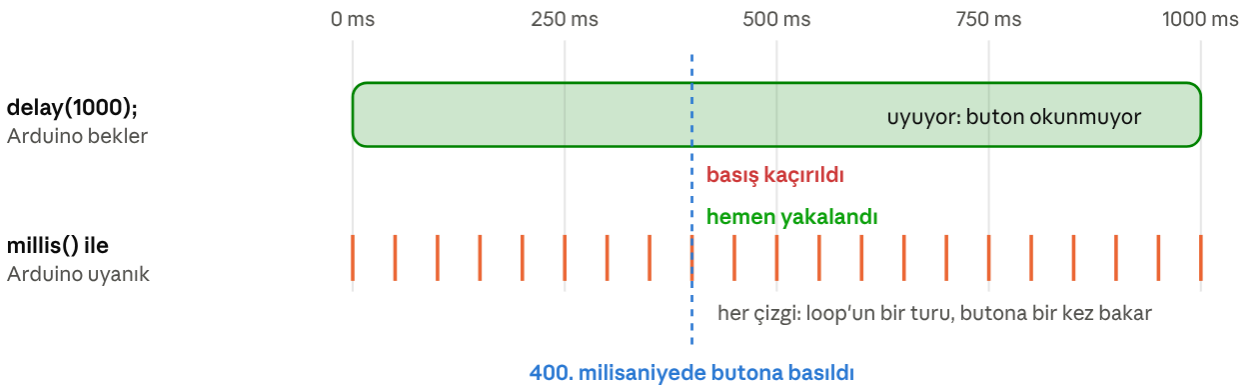
```
int okunan = analogRead(A0); // 0 ... 1023
int parlaklik = map(okunan, 0, 1023, 0, 255); // 0 ... 255
analogWrite(9, parlaklik);
```

`map(değer, eskiMin, eskiMax, yeniMin, yeniMax)` bir sayıyı bir cetvelden diğerine taşır. 512 (yarısı) → 127 (yine yarısı) olur. Basitçe `okunan / 4` yazmak da işe yarardı, çünkü $1024 / 4 = 256$.

6. Zaman: delay ve millis

`delay(ms)` Arduino'yu uyutur; `millis()` ise uyutmadan saati okur. Basit projelerde `delay` yeter, ama aynı anda hem beklemek hem butonu dinlemek gerekiyorsa `millis` şart.

delay beklerken Arduino hiçbir şeyi fark etmez



aynı 1 saniye · iki farklı bekleme yöntemi

Komut	Ne yapar?	Birim
<code>delay(1000)</code>	1000 ms = 1 saniye hiçbir şey yapmadan bekler	milisaniye (ms)
<code>delayMicroseconds(500)</code>	Çok kısa bekler	mikrosaniye (µs), 1 ms = 1000 µs
<code>millis()</code>	Arduino açıldığından beri geçen milisaniyeyi verir	ms
<code>micros()</code>	Aynısını mikrosaniye olarak verir	µs

millis ile beklemek: kronometre mantığı

`millis()` bir kronometre gibidir: "şu an saat kaç?" diye sorarsın. Bekleme yapmak için başlangıç anını not eder, sonra "üzerinden 1 saniye geçti mi?" diye bakarsın. Bakmak anında biter, Arduino bu arada başka işlere de bakabilir.

```

unsigned long sonDegisim = 0;
bool ledAcik = false;

void loop() {
  if (millis() - sonDegisim >= 1000) { // 1 saniye geçti mi?
    sonDegisim = millis();           // yeni başlangıcı not et
    ledAcik = !ledAcik;              // LED'i tersine çevir
    digitalWrite(13, ledAcik);
  }
  if (digitalRead(A4) == LOW) {      // bu arada buton da okunuyor!
    tone(A5, 1000, 50);
  }
}

```

Neden unsigned long ? millis() çok büyük sayılara ulaşır. int en fazla 32.767 tutar; yani 33 saniye sonra taşar ve kod bozulur. unsigned long yaklaşık 49 güne kadar sayar (7. bölüm).

Neden millis() - sonDegisim şeklinde çıkarma? 49 gün sonra millis() sıfırdan başlar. Çıkarma şeklinde yazarsan bu anı da sorunsuz atlatır. Öğrenciler için kural: zamanı her zaman "şimdi eksi başlangıç" diye karşılaştıır.

7. Değişkenler ve tipler

Değişken, üzerinde isim yazılı bir kutudur; içine bir değer koyar, sonra o değeri okur ya da değiştiririz. Tip ise kutunun **boyunu ve içine ne konabileceğini** belirler.

```

int skor = 0;           // tip isim = başlangıç değeri;
skor = skor + 10;       // kutudakini al, 10 ekle, geri koy

```

Büyük kutu büyük sayı tutar, ama RAM'de daha çok yer kaplar

TİP	BOYUT (1 kare = 1 bayt)	NE TUTABİLİR?
bool		true veya false (doğru / yanlış)
char		tek bir karakter: 'A', '7', '#'
byte		0 ... 255
int		-32.768 ... 32.767
unsigned int		0 ... 65.535
long		-2,1 milyar ... 2,1 milyar
unsigned long		0 ... 4,29 milyar (millis() için)
float		ondalıklı sayı: 3.14, -0.5 (6-7 basamak hassas)

TAŞMA: KUTU DOLUNCA KM SAYACI GİBİ BAŞA DÖNER



Arduino UNO'da değişken tipleri ve boyutları

Hangi tipi seçmeliyim?

- Emin değilsen **int** kullan. Sayma, konum, puan gibi işlerin çoğu için yeterli.
- Doğru/yanlış bilgisi için **bool**. Örnek: `bool oyunBitti = false;`
- Zaman için **unsigned long**. `millis()` çok büyük sayılara ulaşır.
- Ondalıklı sayı gerekiyorsa **float**. Ama yavaştır ve tam sayı kadar kesin değildir; gerekmedikçe kullanma.
- RAM'den tasarruf için **byte**. Sadece 0-255 arası yeterliyse.

Tam sayı bölmesi tuzağı

```
int a = 7 / 2;      // a = 3   (küsürat atılır!)
float b = 7 / 2;    // b = 3.0 (önce tam sayı bölünür, sonra float'a konur)
float c = 7 / 2.0;  // c = 3.5 (biri ondalıklı olunca sonuç da ondalıklı)
```

Global ve yerel değişken

Bir değişken nerede tanımlanırsa orada yaşar.

	Global	Yerel
Nerede yazılır?	Fonksiyonların dışında, en üstte	Bir fonksiyonun ya da <code>{ }</code> bloğunun içinde
Kim kullanabilir?	Her yer	Sadece o blok
Ne kadar yaşar?	Arduino açık olduğu sürece	Blok bitince silinir

En sık hata: sayacı `loop()` 'un içinde tanımlamak. `loop()` her turda baştan başladığı için `int sayac = 0;` her turda tekrar sıfırlanır ve sayaç hiç artmaz. Turlar arasında hatırlanması gereken her şey global olmalı.

Sabitler ve diziler

`const int LED_PIN = 13;` değişmeyecek bir değerdir. Kodda 13 yerine `LED_PIN` yazmak okumayı kolaylaştırır; pin değişirse tek yerden düzeltilir.

Dizi, aynı tipten kutulardan oluşan bir raftır:

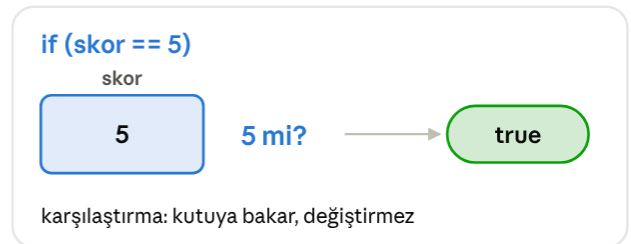
```
int notalar[4] = {262, 330, 392, 523}; // 4 kutulu raf
tone(A5, notalar[0]);                // ilk kutu: 262
tone(A5, notalar[3]);                // son kutu: 523 (4 değil!)
```

Numaralar **0'dan başlar**. 4 kutulu bir dizide `notalar[4]` diye bir kutu yoktur. Arduino bunu kontrol etmez, başka bir değişkenin belleğini okur ve garip hatalar çıkar.

8. Operatörler

Operatörler, değerlerle işlem yapan işaretlerdir. En çok karıştırılan ikili `=` ile `==` 'dir.

Tek = koyar, çift == sorar



`=` ile `==` arasındaki fark

`if (skor = 5)` yazarsan Arduino hata vermez: skor'a 5 koyar ve koşulu hep doğru sayar. Bu hata saatlerce aranabilir; koşullarda her zaman `==` kullan.

Aritmetik

İşaret	Anlamı	Örnek	Sonuç
<code>+</code> <code>-</code>	Toplama, çıkarma	<code>10 - 3</code>	7
<code>*</code>	Çarpma	<code>4 * 5</code>	20
<code>/</code>	Bölme (tam sayıda küsurat atılır)	<code>13 / 8</code>	1
<code>%</code>	Kalan (mod)	<code>13 % 8</code>	5
<code>++</code> <code>--</code>	1 artır, 1 azalt	<code>x++</code>	<code>x = x + 1</code>
<code>+=</code> <code>-=</code>	Ekleyip geri koy	<code>x += 5</code>	<code>x = x + 5</code>

`%` çok işe yarar: `sayi % 2 == 0` sayı çift mi diye sorar; `x % 8` bir sayıyı 0-7 arasında döndürür (8'e gelince başa sarar).

Karşılaştırma (sonuç hep true ya da false)

İşaret	Soru	Örnek
<code>==</code>	Eşit mi?	<code>x == 5</code>
<code>!=</code>	Eşit değil mi?	<code>x != 0</code>
<code><</code> <code>></code>	Küçük mü? Büyük mü?	<code>x < 8</code>
<code><=</code> <code>>=</code>	Küçük ya da eşit mi?	<code>skor >= 10</code>

Mantık (koşulları birleştirir)

İşaret	Okunuşu	Ne zaman true?	Örnek
<code>&&</code>	VE	İkisi de doğruysa	<code>x >= 0 && x < 8</code> (x ekranda mı?)
<code> </code>	VEYA	En az biri doğruysa	<code>x == 0 x == 7</code> (duvarda mı?)
<code>!</code>	DEĞİL	Tersini alır	<code>!oyunBitti</code>

9. Karar ve döngüler: if, for, while

Kod normalde yukarıdan aşağı düz akar. **if** bu akışı **ikiye ayırır**, **for** ve **while** ise bir parçayı **tekrar ettirir**.

if ve for · akış şeması

if/else if/else

```
int isik = analogRead(A0);

if (isik < 300) {
    digitalWrite(13, HIGH);    // karanlık: lamba yansın
} else if (isik < 700) {
    analogWrite(9, 100);       // alacakaranlık: kısık
} else {
    digitalWrite(13, LOW);     // aydınlık: söndür
}
```

Arduino koşulları sırayla dener ve **ilk doğru olanı** çalıştırır; gerisine bakmaz. Hiçbiri doğru değilse **else** çalışır.

for: sayısı belli tekrar

```
for (int i = 0; i < 8; i++) {    // başla; koşul; her turdan sonra
    ledYak(i, 0);
}
```

"Kaç kez tekrar edeceğimi biliyorum" durumunda **for** kullanılır: 8 LED, 10 bip, 3 can gibi.

while: koşul bozulana kadar tekrar

```
while (digitalRead(A4) == HIGH) {    // butona basılana kadar bekle
    // hiçbir şey yapma
}
```

"Ne zaman biteceğini bilmiyorum, bir şey olana kadar" durumunda **while** kullanılır. **Dikkat:** Koşul hiç değişmezse döngü sonsuza kadar sürer ve Arduino "donmuş" gibi görünür.

Döngü	Ne zaman?	Örnek
<code>loop()</code>	Programın kendisi, hep tekrar	Her şey

Döngü	Ne zaman?	Örnek
<code>for</code>	Tekrar sayısı belli	8 satırı doldur
<code>while</code>	Bir şey olana kadar	Butona basılana kadar bekle

10. Kendi fonksiyonunu yazmak

Aynı kod parçasını birkaç yerde kullanıyorsan ona bir isim verip fonksiyon yap. Kod hem kısılır hem de okunur hale gelir. Aslında `pinMode`, `digitalWrite`, `delay` da birilerinin yazdığı fonksiyonlardır.

return eden ve etmeyen fonksiyon

Bir fonksiyonun parçaları

```
// dönüş tipi: int   isim: topla   parametreler: int a, int b
int topla(int a, int b) {
    return a + b;           // sonucu geri ver
}
```

Parça	Anlamı
Dönüş tipi (<code>int</code> , <code>bool</code> , <code>void</code>)	Fonksiyon ne tür bir sonuç verecek? <code>void</code> = hiçbir şey vermez
İsim	Ne iş yaptığını anlatan bir fiil: <code>bipBip</code> , <code>kalpCiz</code> , <code>topla</code>
Parametreler	Fonksiyona dışarıdan verilen değerler. Hiç yoksa <code>()</code> boş kalır
<code>return</code>	Sonucu geri verir ve fonksiyondan hemen çıkar

```

void bipBip(int kez) {
  for (int i = 0; i < kez; i++) {
    tone(A5, 1000, 100);
    delay(200);
  }
}

void loop() {
  bipBip(3);    // 3 kez öter
  delay(2000);
}

```

Neden `setup` ve `loop` önünde `void` yazıyor? Çünkü onlar da sonuç döndürmeyen fonksiyonlardır. Artık Arduino programının aslında fonksiyonlardan oluştuğunu görebilirsin.

11. Seri Monitör, ses ve rastgele sayı

Seri Monitör: Arduino'nun konuşma balonu

Arduino'nun ekranı yok, ama USB kablosu üzerinden bilgisayara yazı gönderebilir. Kodun içinde ne olduğunu görmenin en kolay yolu budur; hata aramada çok işe yarar.

Arduino → USB → Seri Monitör

Komut	Ne yapar?
<code>Serial.begin(9600);</code>	Haberleşmeyi başlatır (<code>setup()</code> içinde, bir kez)
<code>Serial.print(x);</code>	Yazar, alt satıra geçmez
<code>Serial.println(x);</code>	Yazar ve alt satıra geçer

İpucu: Kod beklendiği gibi çalışmıyorsa, şüphelendiğin değişkeni `Serial.println` ile yazdır. "Kodun içine pencere açmak" gibidir.

tone: buzzer'dan ses

Komut	Ne yapar?
<code>tone(pin, frekans);</code>	Sürekli çalar, durdurulana kadar

Komut	Ne yapar?
<code>tone(pin, frekans, süre);</code>	Belirtilen ms kadar çalar
<code>noTone(pin);</code>	Sesi durdurur

Frekans saniyedeki titreşim sayısıdır (Hz). Sayı büyüdükçe ses incelir. Bazı notalar: Do = 262, Re = 294, Mi = 330, Fa = 349, Sol = 392, La = 440, Si = 494, Üst Do = 523.

Dikkat: `tone` sesi başlatır ve hemen bir sonraki satıra geçer; sesin bitmesini beklemez. Arka arkaya notalar çalacaksan araya `delay` koy, yoksa notalar birbirini keser.

```
tone(A5, 262, 200); delay(250); // Do
tone(A5, 330, 200); delay(250); // Mi
tone(A5, 392, 400); delay(450); // Sol
```

random: rastgele sayı

`random(1, 7)` 1 ile 6 arasında bir sayı verir. **Alt sınır dahil, üst sınır hariçtir.** Zar için `random(1, 6)` yazarsan hiç 6 gelmez.

Arduino'nun rastgele sayıları aslında bir formülle üretilir, bu yüzden her açılışta aynı sıra gelir. Bunu önlemek için formüle her seferinde farklı bir başlangıç (tohum) verilir: `randomSeed(analogRead(A0));`. Boştaki bir analog pin rastgele gürültü okur. Ya da butona basılan an kullanılır: `randomSeed(micros());`. Matris projelerinde A0 matrise bağlı olduğu için ikinci yöntemi kullandık.

12. Sık yapılan hatalar

Hata mesajları İngilizce ve korkutucu görünür ama çoğu birkaç basit sebepten çıkar. Mesaj genelde hatanın **bulunduğu satırı ya da bir altını** gösterir; önce o satıra, sonra bir üstüne bak.

Derleme hataları (kod yüklenmez)

Mesaj	Türkçesi	Genelde sebebi
<code>expected ';' before ...</code>	Burada <code>;</code> bekliyordum	Bir önceki satırın sonunda noktalı virgül unutuldu
<code>'xyz' was not declared in this scope</code>	xyz diye bir şey tanıımıyorum	İsim yanlış yazıldı (büyük/küçük harf!) ya da değişken başka bir bloğun içinde tanımlandı

Mesaj	Türkçesi	Genelde sebebi
expected '}' at end of input	Kodun sonunda } bekliyordum	Açılan bir süslü parantez kapanmamış
expected ')' before ...	Burada) bekliyordum	Normal parantez eksik
stray '\342' in program	Programın içinde başıboş bir karakter var	Kod internetten ya da Word'den kopyalandı; eğik tırnak gibi bir karakter geldi. O satırı elle yeniden yaz

Yükleme hataları

Belirti	Sebebi ve çözümü
Port menüsü gri ya da boş	Kablo takılı değil, kablo veri taşımayan bir şarj kablosu ya da CH340 sürücüsü kurulu değil
avrdude: stk500_recv(): programmer is not responding	Yanlış port ya da kart seçili. Araçlar menüsünden kartı "Arduino Uno", portu doğru olanı seç. 0 ve 1 numaralı pinlere bir şey bağlıysa çıkar

Kod yükleniyor ama çalışmıyor

Belirti	Muhtemel sebep
LED çok sönük yanıyor	pinMode(pin, OUTPUT) unutulmuş
Buton rastgele basılı gibi davranıyor	INPUT_PULLUP yerine INPUT kullanılmış, pin havada kalıyor
if koşulu hep çalışıyor	== yerine = yazılmış
Sayaç hiç artmıyor	Değişken loop() 'un içinde tanımlanmış, her turda sıfırlanıyor
Buton basışı bazen kaçıyor	Uzun delay sırasında basılmış; millis kullan (6. bölüm)
Seri Monitörde anlamsız karakterler	Serial.begin hızı ile monitörün hızı farklı

Belirti	Muhtemel sebep
Bir süre sonra garip davranıyor	<code>int</code> taşmış (örneğin <code>millis()</code> <code>int</code> 'e konmuş) ya da dizinin dışına yazılmış

Hata ayıklamada altın kural: Bir seferde tek bir şey değiştir ve tekrar dene. Aynı anda beş şey değiştirirsen hangisinin düzelttiğini (ya da bozduğunu) bilemezsin.